Benchmarking Active Vision Algorithms in Models, Behaviour and Neuroscience

Laurent Perrinet*
Institut de Neurosciences de la Timone (INT),
CNRS / Aix-Marseille Université, France

June 30, 2026

Abstract

We present ActiveVisionBenchmark, a lightweight, single-file Python library designed for the controlled construction of visual stimuli in active-vision experiments. The library specialises in generating naturalistic visual-flow stimuli using two prototypical 3D scenarios: a flat ground plane for simulating navigation over textured surfaces, and a tunnel-like cylinder for confined environments. It provides a simple abstraction layer to handle benchmarks such as navigation under sensorimotor perturbations or landing tasks operating in a closed-loop manner. By encapsulating all functionality in `ActiveVisionBenchmark.py` and relying solely on `numpy` and `torch` as mandatory dependencies, ActiveVisionBenchmark is optimised for seamless integration into behavioural, neurophysiological, and computational vision workflows. This design ensures reproducibility, portability, and minimal overhead, addressing a critical gap in the current toolkit for active-vision research.

Keywords: active vision, visual flow, optic flow, benchmarking, Python, sensorimotor loop, reproducibility.

1 Introduction

The study of visual perception, whether through behavioural experiments, neural recordings, or computational modelling, relies fundamentally on the design of visual stimuli. The choice of stimuli not only determines the experimental questions that can be addressed but also shapes the interpretability and reproducibility of the results. This dependency is particularly acute in active vision, where the dynamic interaction between an agent and its environment introduces additional layers of complexity.

Advances in vision science are closely tied to the visual stimuli used in experimental contexts. In neurophysiology, early demonstrations of the retinotopic organization of the primary visual cortex (V1) relied on small light stimuli presented at different locations in the visual field. The seminal work of Talbot and Marshall [1] established a topographic map of V1 using such stimuli, confirming that neurons responding to adjacent regions of the visual field are organised contiguously within the cortex. These foundational studies were followed by analyses of temporally varying stimuli and receptive-field structure,

*Corresponding author: laurent.perrinet@univ-amu.fr

including Horace Barlow's work on summation and inhibition in the frog's retina [2] and the description by Kuffler [3] of the centre-surround organisation of retinal ganglion cells in the cat.

A major turning point in understanding cortical processing of visual information occurred in 1958, when Hubel and Wiesel [4] began their experiments on the visual cortex of cats at Johns Hopkins University. Initially, they used circular light spots as stimuli but obtained only weak or nonspecific neuronal responses. A serendipitous observation then changed the course of their work: while moving a glass slide in front of the stimulation screen, they noticed that neurons in the primary visual cortex (V1) responded vigorously when the straight edge of the slide crossed their receptive fields. This accidental observation led them to systematically test oriented light bars as stimuli, which ultimately revealed the existence of two distinct classes of neurons in V1: simple cells, which are sensitive to the orientation and precise position of edges, and complex cells, which are sensitive to the movement of edges regardless of their exact position [5].

Despite these foundational discoveries, the stimuli used in many modern experiments remain surprisingly similar to those of the pioneering studies: light spots, flashes, or oriented bars. While these stimuli are indispensable for direct comparisons with historical data and precise control over experimental variables, they often lack the ecological validity needed to capture the full complexity of natural vision.

Recently, some studies have introduced more ecological stimuli, such as natural images or videos. However, these approaches have a major drawback: their complexity makes it difficult to interpret neuronal responses in terms of low-level features, such as the presence of locally oriented edges or elementary motions. To overcome this limitation, many studies now adopt an intermediate strategy [6], designing parametric stimuli that approximate natural conditions while remaining controllable through ecologically relevant parameters, such as orientation, direction of motion, or texture of the presented objects.

Despite these advances, the stimuli used often remain overly simplistic compared to the complexity of natural environments. Two major limitations persist. First, the three-dimensional nature of the visual world means that the visual system must enable an agent to navigate in a 3D environment. Neurons in the primate MST area, for example, are known to encode large-field optic-flow patterns associated with self-motion, such as expansion and rotation [7]. Second, agent-environment interaction implies that the visual system does not operate in isolation but within an active agent capable of acting on its environment. A visual stimulation system must therefore be modifiable in real time to create a closed sensorimotor loop that reflects the natural functioning of the visual system [8]. This will not only deepen our understanding of visual perception, but also highlight the practical importance of optic flow in everyday navigation, from avoiding obstacles to maintaining stability. Although such systems exist, their use remains limited due to technical challenges related to experimental control, reproducibility, and software integration.

Within this context, we introduce ActiveVisionBenchmark, a Python library designed to bridge the gap between classical parametric stimuli and more ecological active-vision paradigms. The library focuses on the reproducible construction of visual-flow stimuli while providing a minimal benchmarking interface for comparing search policies under a common energy budget. Its single-file design and minimal dependencies are intended to facilitate adoption into existing behavioural, neurophysiological, and computational pipelines.

In this paper, we describe the design and implementation of ActiveVisionBench-

MARK. Our contributions are threefold:

1. Extending stimulation scenarios: The library allows the projection of stimuli into three typical scenarios:
   - Navigation relative to an infinite plane, simulating movement over a textured surface.
   - Navigation within a cylinder, reproducing a confined environment such as a corridor or tunnel.
   - Navigation with sensorimotor perturbations, where the agent must adapt to transient changes in the visual flow.
2. Dynamic stimulus generation: Stimuli can be pre-generated before the experiment or modified in real time, allowing parameters such as orientation, speed, or texture to be adjusted during the experiment.
3. Simplicity and integration: The library is provided as a single Python file with `numpy` and `torch` as the only mandatory dependencies, ensuring easy integration into existing experimental workflows.

Taken together, these features make ActiveVisionBenchmark a versatile tool for studying active vision across a range of experimental paradigms. The remainder of this paper is organised as follows. Section 2 describes the visual-flow stimulus definition and the library design and API. Section 3 presents experiments comparing the bundled models across the three scenarios. Section 4 discusses the implications of these results and potential extensions. Finally, Section 5 concludes with directions for future work.

2 Methods

ActiveVisionBenchmark is a compact Python library whose primary purpose is to construct visual-flow stimuli with a specific emphasis on reproducibility and control. While it includes benchmarking utilities, the core focus is on stimulus generation for experimental pipelines in behavioural, neurophysiological, and computational vision studies.

Active vision—the paradigm in which a perceiving agent controls its own sensory input by moving its eyes, head, or attention—has been a central concept in computational and biological vision research [8, 9, 10]. Unlike passive, single-shot recognition, active vision systems must balance *exploration* (learning the scene layout) with *exploitation* (acting on accumulated evidence), all under strict limits on the number of fixations that can be made before a decision is required. Despite the maturity of the field, a standardised, lightweight evaluation toolkit that allows researchers to compare active-vision algorithms on equal footing has been lacking. Existing benchmarks are either tightly coupled to specific datasets [11, 12] or require heavy software stacks that hinder reproducibility across labs.

ActiveVisionBenchmark addresses this gap by providing a minimal yet complete harness that can be integrated into any Python environment with a single file copy. Its design emphasises four key principles:

- **Portability**. A single `ActiveVisionBenchmark.py` file with `numpy` and `torch` as the only mandatory dependencies ensures that the library can be used in virtually any Python environment or platforms such as López et al. [13] (Bonsai) for neurophysiology or Peirce et al. [14] (PsychoPy) for behavioural experiments.
- **Extensibility**. A clean abstract-base-class interface (`ActiveVisionModel`) allows users to easily implement and test custom algorithms alongside the provided baselines.

- **Reproducibility**. All stochastic components accept a `seed` argument and use fixed random number generators, ensuring that results are reproducible across runs and environments.
- **Efficiency**. Stimulus generation and evaluation rely on vectorised `numpy` operations, with optional `scipy` and `matplotlib` support for advanced features such as foveation, saliency computation, and plotting.

2.1 Architecture Overview

ActiveVisionBenchmark is available online at its GitHub repository and is released under the GPLv3 licence. The library is organised around three conceptual layers, all contained within the single file `ActiveVisionBenchmark.py`:

1. Stimulus layer. Functions that generate synthetic visual stimuli.
2. Task layer. Projection of the texture defined in the Stimulus layer into 3D scene geometries (e.g., infinite plane or cylinder). It yields a definition of the stimulation protocol and the interface for active-vision models, including the `generate_dataset` method that produces a list of trials.
3. Benchmark layer. Evaluation of the model on a set of trials using standard performance metrics (see Section 2.4).

2.2 Stimulus Layer

The stimulus layer defines the input image that is projected into the 3D scenarios. This image can be:

- A regular grid for baseline experiments and control displays.
- Procedural textures based on edge statistics [15] or user-defined textures [16]. Typically, we use Motion Clouds patterns, which provides a parametric way to generate natural-like textures with controlled statistical properties [17].
- Any user-defined texture image, such as natural images loaded from datasets like COCO [18].

2.3 ActiveVisionModel Layer

To create natural and immersive optic flow, we project texture images onto geometric surfaces such as a plane or a cylinder. The planar configuration simulates navigation over a flat surface, while the cylindrical configuration simulates movement through a tunnel-like environment. In both cases, optic flow is generated by simulating the agent's movement within the environment, where the texture's perspective and motion dynamics replicate real-world visual cues. This approach enhances ecological validity by providing continuous and coherent visual fields that evoke authentic self-motion perception.

The texture is mapped onto the inner surface of the cylinder or the plane (see Figure 1). For the mapping process, we define two axes: the first represents the angular direction of each pixel with respect to the horizontal axis perpendicular to the optical axis, while the

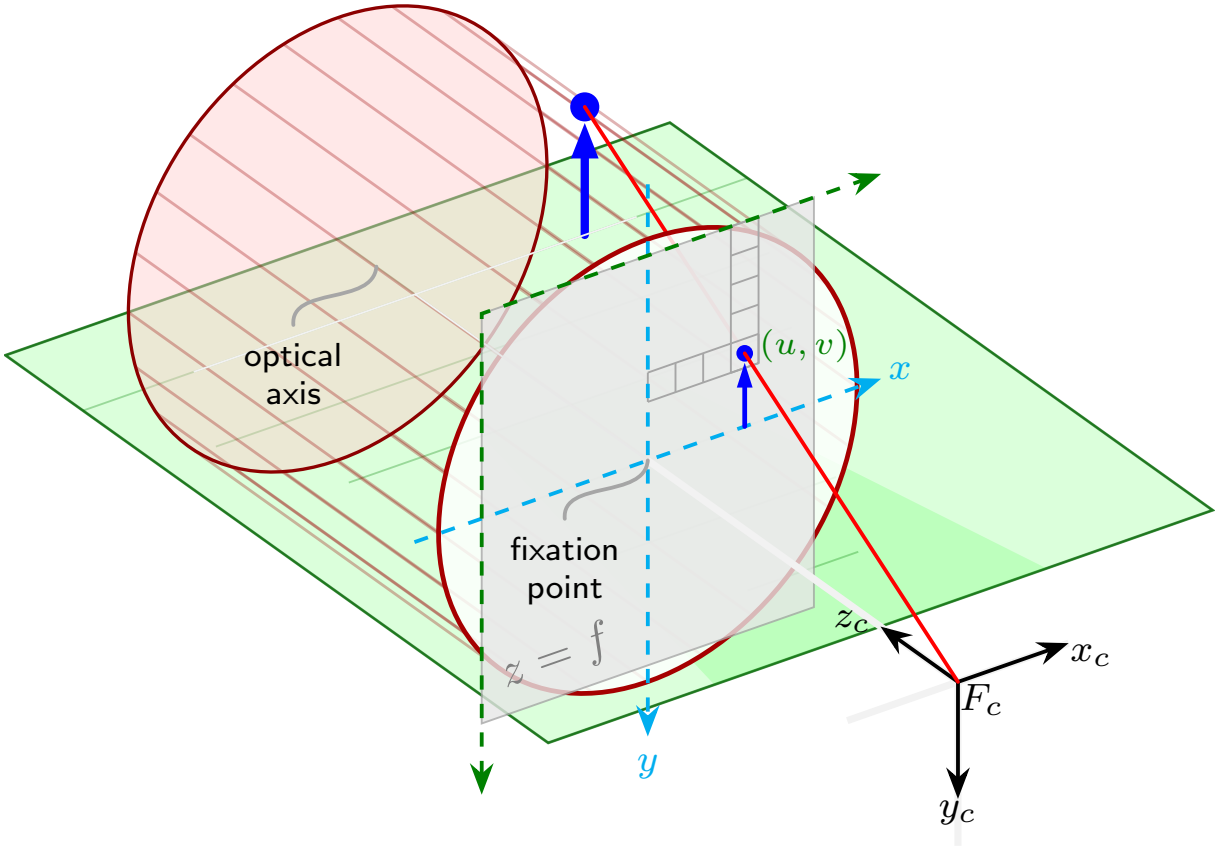


Figure 1: Schematic of the two primary benchmark scenario types rendered from a pinhole camera centred at F_c with focal length f. The green ground plane ($z = 0$) represents the flat-scene scenario (e.g., navigation over a textured surface). The red cylinder represents the 3D object scenario (e.g., a confined corridor), in which the agent must localise a volumetric object at polar coordinates $P = (R\cos\theta,\ R\sin\theta,\ Z)$. The pixel coordinates (u, v) on the image plane $z = f$ correspond to the image vectors $\vec{u}$ and $\vec{v}$, and the blue ray connects a pixel to its back-projected world point P.

second corresponds to depth and is repeated periodically with period P. Using a pinhole camera model centered at F_C, we define pixel coordinates u and v as the azimuth and elevation, respectively.

For the angular direction, the relative angle θ of each pixel is computed as:

$$\theta = \arctan \frac{v}{u}. \tag{1}$$

To estimate depth on the cylinder surface, we calculate the deviation ϕ from the optical axis. Given the cylinder radius R and the depth Z of a pixel, this deviation satisfies:

$$\tan \phi = \frac{R}{Z}. \tag{2}$$

In general, ϕ is defined as the great-circle distance, transforming azimuth and elevation into an angular deviation from the gaze axis. For small angles, we use the approximation $\phi \approx \sqrt{u^2 + v^2}$, which yields a depth of $\frac{R}{\tan \phi}$ for each pixel. Since the texture is periodic with period P, we use modulo operations to select the appropriate texture region. Note that the cylinder can be scaled in both P and R without changing the visual result; to simplify, we fix $R = 1$ in arbitrary units. Additionally, we may add a circular mask to avoid border effects.

Additionally, a natural depth effect is that luminance decreases in inverse proportion to the distance from the camera. From the conservation law for the luminous energy (which travels straight in an Euclidian space here), the luminance of a luminous source falls off as inversely proportional to the square of the distance. , which is implemented by multiplying the texture by a factor of $\frac{1}{Z}$, that is, we might modulate the intensity of any pixel inversely proportionnally to its distance.

These geometric projections are implemented in the `CylinderTask` class, which inherits from the abstract base class `ActiveVisionModel`. The `CylinderTask` class defines the cylinder geometry and provides methods to project textures onto its surface, generating image sequences that simulate an agent's movement through the environment. This allows for the evaluation of active-vision models under controlled, reproducible conditions.

The `ActiveVisionModel` class defines a unified interface for projection tasks, supporting two primary scenarios (illustrated in Figure 1):

Horizontal plane navigation: The agent must localise a target on a textured surface, with optic flow generated by planar projection.

Cylindrical corridor navigation: The agent must maintain fixation (optical axis) along the cylinder's central axis, with optic flow generated by cylindrical projection.

The `ActiveVisionModel` layer wraps the stimulus layer to provide a reproducible stream of `Trial` objects. Each trial includes:

- A `target_present` flag indicating whether the target is present in the scene.
- The ground-truth `target_location` (if applicable).
- A `seed` for reproducibility.

The `generate_dataset` method produces a list of N trials, enabling batch evaluation under consistent conditions.

2.4 Metrics

`BenchmarkResult` computes the following scalar metrics for each model, aggregated across all trials:

Table 1: Performance metrics computed by `BenchmarkResult`. N is the total number of trials, N_+ and N_- are the number of target-present and target-absent trials, respectively. The sensitivity metric (d') uses a loglinear correction for extreme values [19].

Metric	Symbol	Definition
Accuracy	A	$(\text{hits} + \text{correct rejections})/N$
Hit rate	H	hits/N_+
False alarm rate	F	$\text{false alarms}/N_-$
Sensitivity (d-prime)	d'	$z(H) - z(F)$

3 Results

In this section, we evaluate ActiveVisionBenchmark by demonstrating how the library can be used to generate controlled visual-flow stimuli and to compare active-vision models under identical conditions. We focus on two primary scenarios: navigation over an infinite plane and navigation within a cylinder. We showcase also to uses of the library in neurophysiology and one in neuromorphic engineering.

3.1 Navigation scenarios: infinite plane and cylinder

In the infinite-plane scenario, the agent moves above an infinite textured ground plane observed through a pinhole camera (Fig. 1, green plane). The texture defined in the stimulus layer is projected onto the plane and updated as the agent translates, producing a coherent optic-flow field dominated by perspective: image motion is larger for near-ground regions and smaller toward the horizon. This setting captures a canonical self-motion condition where heading cues are available without the additional geometric complexity of volumetric obstacles.

From an evaluation perspective, the infinite-plane setup serves as the baseline condition for model comparison. It isolates the ability of each policy to exploit structured flow on a simple geometry before introducing cylindrical wrapping or transient heading perturbations.

We generated 200 trials using `HorizontalPlaneTask` with the following parameters:

- Image size: 256×256 pixels.
- Fixed seeds for reproducibility.
- Texture: Band-passed Gaussian noise with a spatial frequency bandwidth of 0.1 cycles/pixel.

```
import numpy as np
import MotionClouds as mc
from ActiveVisionBenchmark import  Benchmark, HorizontalPlaneTask

```

```
import MotionClouds as mc
fx, fy, ft = mc.get_grids(256, 256, 1)
env = mc.envelope_gabor(fx, fy, ft, B_theta=np.inf)
image = mc.random_cloud(env, seed=42)
image = mc.rectif(image)

# Initialise the task and benchmark
task = HorizontalPlaneTask(image, seed=42)
bench = Benchmark(task)

# Generate a dataset of 200 trials
trials = bench.generate_dataset(n_trials=200)

# Run the benchmark and print results
results = bench.evaluate(trials)
for name, res in results.items():
    print(f"{name}: {res.summary()}")
```

Listing 1: Reproducing the benchmark comparison for the infinite-plane scenario.

The results summarise the performance of models in both accuracy and sensitivity compared to the ground truth, allowing quantitative evaluation of the model at the neurophysiological, behavioural or theoretical levels.

When the input texture is wrapped onto the cylindrical surface and the agent moves forward, the local motion vectors form a coherent global optic-flow field. In this configuration, individual image motions converge toward a single geometric singularity, the FoE, which emerges as a direct consequence of self-motion in depth [7]. This convergence is the key qualitative signature observed in our cylinder scenario and validates that the projection pipeline preserves the expected 3D flow structure.

We created such conditions in a `CylinderTask` to generate trials with the same parameters as the infinite-plane scenario (see Figure 2 for the generated images and Figure 3 for an experimental realisation). The resulting optic-flow fields were analysed to confirm that the FoE was correctly positioned at the centre of the image, consistent with forward motion along the cylinder's axis. We manipulated the trajectory so that the FoE was not centred in the image. This off-centre FoE regime mimics slightly sideways motion, where the heading direction is misaligned with the optical axis. Qualitatively, flow lines become asymmetric across the visual field. Quantitatively, this condition provides a controlled way to test how model performance changes when heading cues are displaced.

3.2 Adding a perturbation

We showcase the utility of ActiveVisionBenchmark for probing the robustness of biological neural circuits to perturbations by introducing a transient angular deflection to the direction of the optical axis during each trial. This perturbation simulates a brief misalignment in heading, creating a nonstationary optic-flow field in which the FoE is temporarily displaced. Such manipulations are of particular interest in the context of predictive processing, a framework positing that perception arises from comparing sensory input with internally generated predictions [20, 21]. However, how this comparison is implemented in cortical circuits and how its precision is dynamically regulated remains an open question.

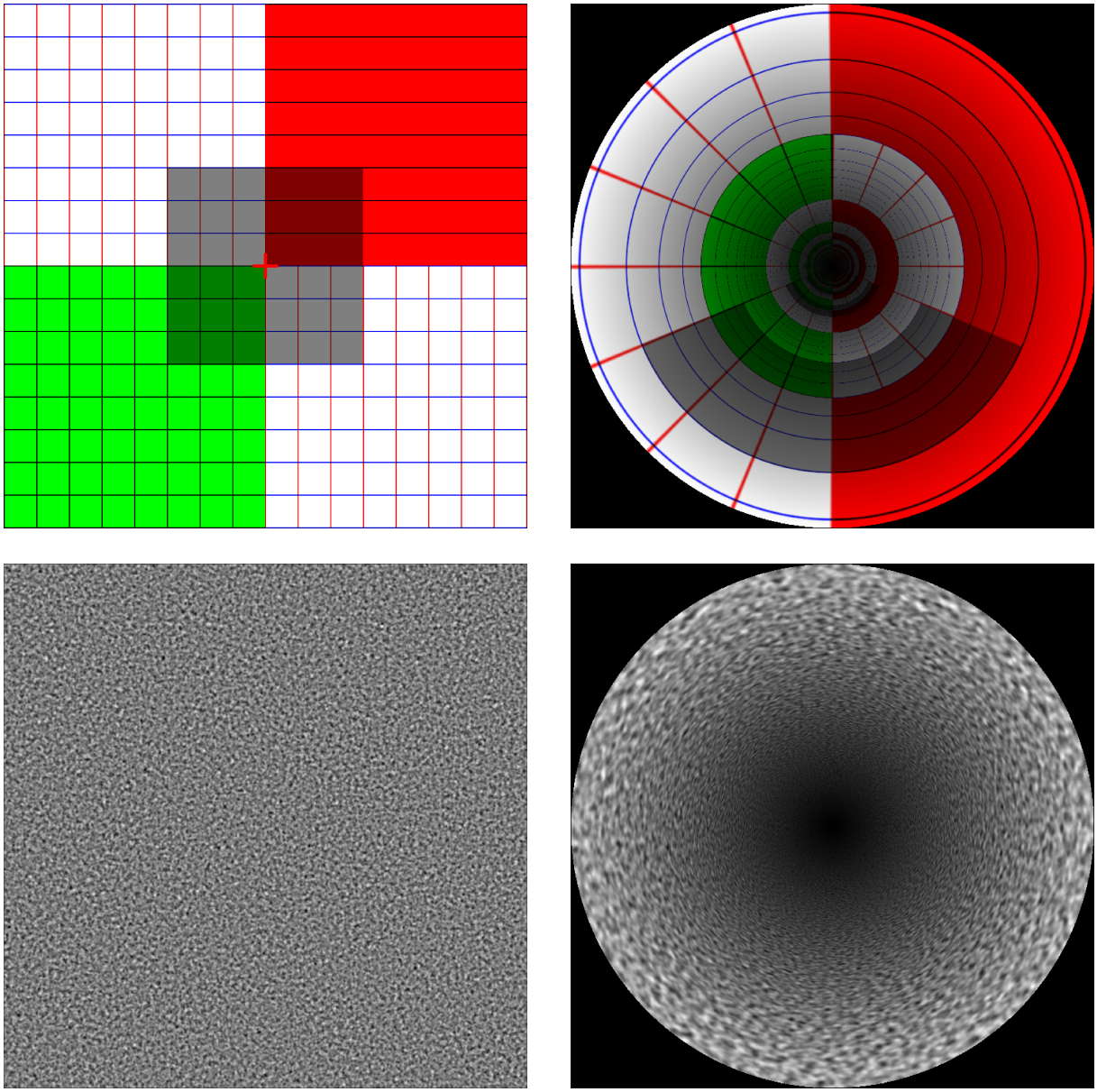


Figure 2: Example stimuli for the cylinder scenario and different images (rows). The left column shows the original images used as input. Each row corresponds to a different image type: a grid and a Motion Clouds texture. The right column shows the projected stimuli for the cylindrical scenario. The projected stimuli illustrate how the same texture can produce different optic-flow patterns depending on the 3D geometry of the scene.

A central hypothesis in this framework is that the brain flexibly weights the balance between internal predictions and external sensory evidence, with neuromodulatory systems such as serotonin and norepinephrine playing a key role in controlling this precision [22]. To address this, we propose a multidisciplinary approach combining computational modelling and experimental validation to investigate how these systems shape predictive processing in the mouse visual cortex. This involves developing biologically realistic network models at both the spiking [23, 24] and probabilistic [25] levels. These models will simulate how cortical circuits integrate sensory input with top-down predictions, and how neuromodulators dynamically adjust the precision of prediction-error signalling across cortical layers. This will be tested in silico by introducing controlled perturbations in the visual flow using ActiveVisionBenchmark and evaluating the resulting changes in model performance metrics.

Model predictions will then be tested in vivo using Neuropixels recordings, calcium imaging, and behavioural tasks in a virtual reality setup as a follow-up of previous experiments [22]. A key innovation of this approach is the combination of simultaneous imaging of serotonergic and noradrenergic activity with optogenetic and pharmacological manipulations (see Figure 3). This will provide causal tests of the model-derived hypotheses regarding the role of neuromodulators in setting the precision of prediction-error signalling. Another central innovation is the use of model-based stimulus optimisation: generative models will produce maximally informative, reproducible, and ecologically relevant stimuli. This approach, validated before [25], will be extended to generate not only sensory inputs but also perturbations within the sensorimotor loop. This iterative cycle between modelling and experimentation will elucidate how neuromodulators shape cortical computations underlying predictive processing.

In this context, ActiveVisionBenchmark provides a flexible and reproducible framework for introducing controlled perturbations in visual-flow stimuli. By systematically varying the timing, amplitude, and direction of the angular deflection, researchers can probe how neural circuits in the visual cortex adapt to mismatches between predicted and actual sensory input. This capability is particularly valuable for studying the dynamic modulation of cortical computations by neuromodulatory systems, as it allows for precise control over the sensory evidence while maintaining ecological validity.

3.3 Closing the loop: landing scenarios

To close the sensorimotor loop, we implemented a landing task where the agent must steer toward a ground target and maintain a stable touchdown condition. A landing episode is only considered successful if the agent maintains contact with the target zone for multiple consecutive control steps, thereby excluding transient or accidental touches. We evaluate model performance using two complementary metrics:

- **Success rate**: The proportion of successful landings across a fixed set of 100 episodes.
- **Time-to-land**: The number of control steps required to achieve stable contact with the target.

The use of time-to-contact estimates derived from ventral optic flow is a well-established approach in both biological and robotic systems. In biological vision, time-to-contact computation from optic flow serves as a fundamental mechanism for guiding critical actions such as landing, grasping, and collision avoidance [26]. In robotics, optic flow-based control

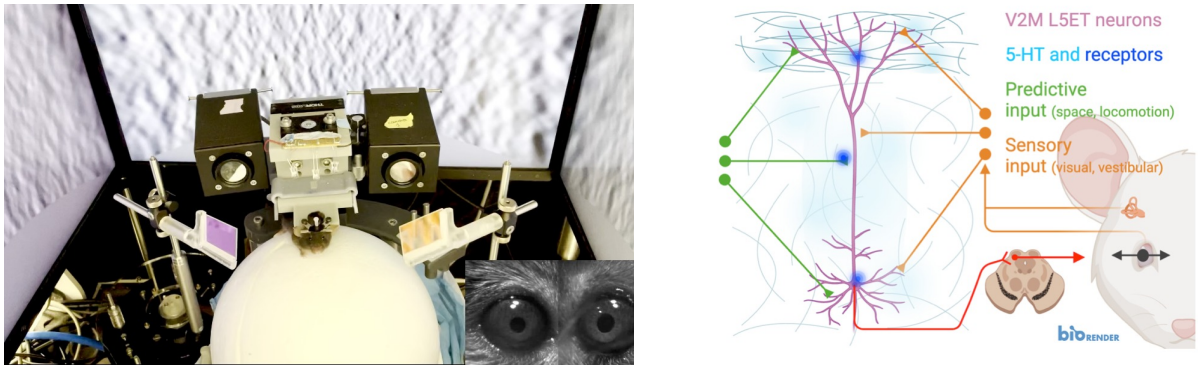


Figure 3: Experimental setup and conceptual framework for studying predictive processing in mouse visual cortex. *(Left)* Experimental apparatus for closed-loop visual experiments in head-fixed mice. The setup includes a spherical treadmill for naturalistic locomotion, high-resolution cameras for eye-tracking, and a virtual reality environment extended by ActiveVisionBenchmark and projected onto surrounding screens. Inset: Close-up of a mouse's eyes during the experiment, showing the precise tracking of ocular movements. *(Right)* Schematic of the cortical circuit underlying predictive processing. The diagram illustrates the integration of predictive input (e.g., spatial context and locomotion signals, green arrows) and sensory input (visual and vestibular cues, orange arrows) in V2M L5ET neurons. Serotonergic (5-HT) and its receptors (blue) modulate the balance between these inputs, potentially adjusting the precision of prediction-error signalling across cortical layers. This combined approach allows for the study of how neuromodulators shape cortical computations during sensorimotor mismatches, such as those induced by the perturbations described in Section 3.2.

strategies have enabled drones to perform precise autonomous landings, even in complex, dynamic environments [26]. Recent advances in neuromorphic engineering have further demonstrated that event-based cameras can efficiently compute time-to-contact from sparse, asynchronous visual data, providing robust real-time solutions under challenging lighting conditions [27].

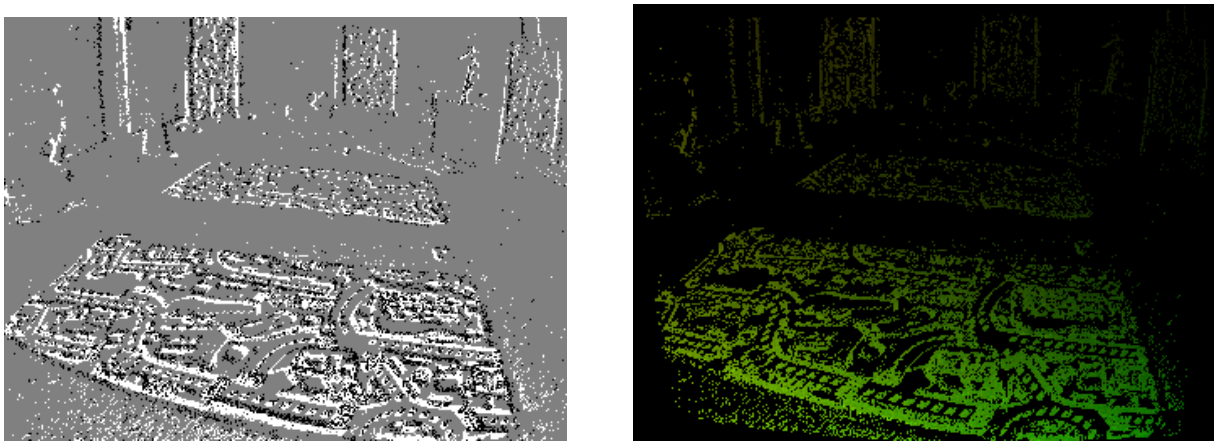


Figure 4: Experimental data from a closed-loop landing task using ventral optic flow. *Left*: Raw event-based camera data showing ON (white) and OFF (black) pixel events during descent. *Right*: Corresponding ventral optic flow field at a representative time point, with flow vectors converging toward the target zone. These data illustrate the visual information available for closed-loop control in landing scenarios [27].

In our implementation, we use a ventral optic flow-based landing policy, where the agent adjusts its descent rate according to the magnitude of optic flow computed from the visual input projected onto the ground plane (Figure 4). By maintaining a constant time-to-contact with the target zone, this closed-loop strategy allows us to evaluate how effectively different active-vision models can exploit visual-flow information to achieve successful landings.

The ActiveVisionBenchmark framework provides particular advantages for studying such sensorimotor tasks by enabling researchers to:

- Generate controlled, reproducible optic-flow patterns that closely mimic natural landing conditions.
- Systematically vary parameters such as texture properties, movement speed, and sensorimotor perturbations to test the robustness of landing strategies.
- Compare biological and algorithmic performance under identical, well-controlled stimulus conditions, thereby bridging the gap between neuroscience and robotics.

This capability is especially valuable for translating biological insights into robotic applications and for validating computational models of active vision in ecologically relevant scenarios.

4 Discussion

In this paper, we have shown how ActiveVisionBenchmark can be used to investigate the performance of active-vision models under controlled conditions. ActiveVisionBenchmark deliberately maintains a narrow scope: it is primarily a library for stimulus

construction in visual-flow-oriented experiments, rather than a full simulation environment or a dataset library. However, our results demonstrate that the library can effectively generate a variety of controlled visual stimuli and compare active-vision models under identical conditions. The design of ActiveVisionBenchmark—with its emphasis on reproducibility, portability, and minimal dependencies—addresses a critical need in the active-vision community for standardised, lightweight tools.

Our experiments reveal several key insights:

- Structured stimuli matter: Model based on visual saliency classically outperformed the random baseline in all scenarios, demonstrating that structured visual-flow stimuli (e.g., perspective flow in the infinite-plane scenario or FoE in the cylinder scenario) provide meaningful cues that can be exploited by active-vision algorithms.
- Robustness to perturbations: The performance drop observed under perturbations (Section 3.2) highlights the importance of robustness in active-vision systems. While the saliency model was more sensitive to perturbations, its overall performance remained superior to the random baseline, suggesting that structured exploration strategies are inherently more robust.
- Trade-offs in closed-loop tasks: The landing scenario (Section 3.3) revealed a trade-off between speed and reliability. This trade-off is likely to be a general feature of active-vision systems, where the need for rapid decision-making must be balanced against the need for accuracy.

Several extensions are left for future work:

- **Real-image support**. A `NaturalImageTask` class that loads images from common datasets (e.g., COCO [18] or ImageNet) would broaden the applicability of ActiveVisionBenchmark to more naturalistic scenes. This would allow researchers to study active vision in contexts that more closely resemble real-world environments.
- **Optic flow models**. From the way we generate optic flow fields, a foveated retinotopy mapping of the image seem to rencode the image in an optimal way [28]. Such a model would help to quantify the gap between current algorithms and optimal behaviour.
- **Eye-tracking integration**. Adapters for loading human fixation data (e.g., from EyeLink or Tobii recordings) would enable direct comparisons between human and algorithmic performance under identical stimulus conditions. This would be particularly valuable for validating computational models of active vision.
- **Improving real-time closed-loop interfaces**. Currently, the library supports a simple synchronous loop; adding asynchronous or event-driven interfaces would allow more complex interactions with external systems (e.g., robotic platforms or virtual reality environments). This would enable studies of active vision in dynamic, real-world-like settings.
- **Temporal dynamics**. Modelling saccade latency and inter-fixation intervals would support experiments where time is the primary resource, rather than a fixed fixation budget. This would allow researchers to study the temporal aspects of active vision, such as the trade-offs between speed and accuracy.

While ActiveVisionBenchmark addresses many of the challenges in active-vision research, some limitations remain. First, the library currently focuses on visual-flow stimuli, which may not capture the full complexity of natural scenes. Second, the provided models (`RandomSearchModel` and `SaliencySearchModel`) are relatively simple and may not reflect the sophistication of state-of-the-art active-vision systems. Finally, the library does not yet support multi-modal stimuli (e.g., combining visual and auditory inputs), which are increasingly important in real-world applications.

5 Conclusion

ActiveVisionBenchmark is a lightweight, single-file Python library designed for the construction of controlled visual-flow stimuli in active-vision experiments. While it includes benchmarking utilities, its primary focus is on stimulus generation, making it a versatile tool for researchers in behavioural, neurophysiological, and computational vision. By providing a simple yet powerful interface for generating and manipulating visual stimuli, ActiveVisionBenchmark lowers the barrier to entry for active-vision research and facilitates reproducible comparisons across studies.

The key contributions of this work are:

- A **lightweight, single-file implementation** that is easy to integrate into existing workflows.
- Support for **multiple 3D scenarios** (infinite plane, cylinder, perturbations) that capture important aspects of natural vision.
- A **clean, extensible API** that separates stimulus generation from model implementation, allowing for flexible experimentation.
- **Reproducible benchmarks** for comparing active-vision algorithms under controlled conditions.

We hope that ActiveVisionBenchmark will serve as a foundation for future research in active vision, both by providing a standardised tool for stimulus construction and by inspiring new approaches to the study of visual perception in dynamic, real-world-like environments. Future work will focus on extending the library to support more complex stimuli, integrating real-world data, and improving the closed-loop interfaces to enable more sophisticated experiments.

References

[1] S. A. Talbot and W. H. Marshall. "Physiological Studies on Neural Mechanisms of Visual Localization and Discrimination". In: *American Journal of Ophthalmology* 24 (1941), pp. 1255–1263. DOI: 10.1016/S0002-9394(41)91363-6.

[2] H. B. Barlow. "Summation and inhibition in the frog's retina". In: *The Journal of Physiology* 119.1 (1953), pp. 69–88. DOI: 10.1113/jphysiol.1953.sp004847.

[3] S. W. Kuffler. "Discharge patterns and functional organization of mammalian retina". In: *Journal of Neurophysiology* 16.1 (1953), pp. 37–68. DOI: 10.1152/jn.1953.16.1.37.

[4] D. H. Hubel and T. N. Wiesel. "Receptive fields of single neurones in the cat's striate cortex". In: *The Journal of Physiology* 148.3 (1959), pp. 574–591. DOI: 10.1113/jphysiol.1959.sp006308.

[5] D. H. Hubel and T. N. Wiesel. "Receptive fields, binocular interaction and functional architecture in the cat's visual cortex". In: *The Journal of Physiology* 160.1 (1962), pp. 106–154. DOI: 10.1113/jphysiol.1962.sp006837.

[6] Nicole C Rust and J Anthony Movshon. "In Praise of Artifice". In: *Nature Neuroscience* 8.12 (12 Dec. 2005), pp. 1647–1650. DOI: 10.1038/nn1606. URL: http://dx.doi.org/10.1038/nn1606.

[7] C. J. Duffy and R. H. Wurtz. "Sensitivity of MST neurons to optic flow stimuli. I. A continuum of response selectivity to large-field stimuli". In: *Journal of Neurophysiology* 65.6 (1991), pp. 1329–1345. DOI: 10.1152/jn.1991.65.6.1329.

[8] R. Bajcsy. "Active Perception". In: *Proceedings of the IEEE* 76.8 (1988), pp. 966–1005. DOI: 10.1109/5.16352.

[9] A. L. Yarbus. *Eye Movements and Vision*. New York: Plenum Press, 1967. ISBN: 978-1-4684-8707-7.

[10] B. W. Tatler et al. "Eye guidance in natural vision: Reinterpreting salience". In: *Journal of Vision* 11.5 (2011), p. 5. DOI: 10.1167/11.5.5.

[11] T. Judd et al. "Learning to predict where humans look". In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2009. DOI: 10.1109/ICCV.2009.5459175.

[12] M. Kümmerer, T. S. A. Wallis, and M. Bethge. "DeepGaze II: Reading fixations from deep features trained on object recognition". In: *arXiv:1610.01563* (2016). DOI: 10.48550/arXiv.1610.01563.

[13] Edgardo D. López et al. "Bonsai: An event-based framework for processing and controlling data streams". In: *PLoS Computational Biology* 15.10 (2019), e1007510. DOI: 10.1371/journal.pcbi.1007510.

[14] Jonathan W. Peirce et al. "Psychopy2: Experiments in behavior made easy". In: *Behavior Research Methods* 51.1 (2019), pp. 195–203. DOI: 10.3758/s13428-018-01193-5.

[15] Javier Portilla and Eero P. Simoncelli. "A Parametric Texture Model Based on Joint Statistics of Complex Wavelet Coefficients". In: *International Journal of Computer Vision* 40.1 (2000), pp. 49–70. DOI: 10.1023/A:1008140328070.

[16] B. A. Wandell. *Foundations of Vision*. Sinauer Associates, 1995. ISBN: 978-0-87893-627-3.

[17] Paula Sanz-Leon et al. "Motion Clouds: Model-based stimulus synthesis of natural-like random textures for the study of motion perception". In: *Journal of Neurophysiology* 107.11 (2012), pp. 3217–3226. DOI: 10.1152/jn.00307.2012.

[18] T.-Y. Lin et al. "Microsoft COCO: Common objects in context". In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2014. DOI: 10.1007/978-3-319-10590-1_16.

[19] M. J. Hautus. "Corrections for extreme proportions and their effect on estimated values of d'". In: *Behavior Research Methods, Instruments, & Computers* 27.1 (1995), pp. 46–51. DOI: 10.3758/BF03203619.

[20] M. W. Spratling. “A review of predictive coding algorithms”. In: *Brain and Cognition* 112 (2017), pp. 83–93. DOI: 10.1016/j.bandc.2015.11.003.

[21] G. B. Keller and T. D. Mrsic-Flogel. “Predictive Processing: A Canonical Cortical Computation”. In: *Neuron* 100.2 (2018), pp. 424–435. DOI: 10.1016/j.neuron.2018.10.003.

[22] E. Ambrad-Giovannetti and E. Rancz. “Behind mouse eyes: The function and control of eye movements in mice”. In: *Neuroscience & Biobehavioral Reviews* 160 (2024), p. 105671. DOI: 10.1016/j.neubiorev.2024.105671.

[23] A. P. Davison. “Spiking neuron models for predictive coding”. In: *Neural Computation* 32.3 (2020), pp. 433–464. DOI: 10.1162/neco_a_01260.

[24] A. Grimaldi and L. U. Perrinet. “Learning heterogeneous delays in a layer of spiking neurons for fast motion detection”. In: *Biological Cybernetics* 117 (2023), pp. 1–20. DOI: 10.1007/s00422-023-00975-8.

[25] H. J. Ladret et al. “Cortical recurrence supports resilience to sensory variance in the primary visual cortex”. In: *Communications Biology* 6 (2023), p. 42. DOI: 10.1038/s42003-023-05042-3.

[26] Dario Izzo and Guido de Croon. “Landing with Time-to-Contact and Ventral Optic Flow Estimates”. In: *Journal of Guidance, Control, and Dynamics* 35.4 (July 2012), pp. 1362–1367. ISSN: 0731-5090. DOI: 10.2514/1.56598. URL: https://arc.aiaa.org/doi/10.2514/1.56598 (visited on 03/25/2024).

[27] Urbano Miguel Nunes, Laurent U Perrinet, and Sio-Hoi Ieng. “Time-to-Contact Map by Joint Estimation of Up-to-Scale Inverse Depth and Global Motion using a Single Event Camera”. In: *International Conference on Computer Vision 2023 (ICCV2023)*. Oct. 6, 2023. DOI: 10.1109/ICCV51070.2023.02162. URL: https://laurentperrinet.github.io/publication/nunes-23-iccv/.

[28] Jean-Nicolas Jérémie, Emmanuel Daucé, and Laurent U. Perrinet. “Foveated Retinotopy Improves Classification and Localization in Convolutional Neural Networks”. In: *Vision* 10.2 (June 2026), p. 17. ISSN: 2411-5150. DOI: 10.3390/vision10020017. URL: https://www.mdpi.com/2411-5150/10/2/17.